
THE GEORGE WASHINGTON UNIVERSITY

WASHINGTON, DC

MS in Data Science

DATS 6303 - Deep Learning - Spring 2025
Final Report

Driver Drowsiness Detection

Presented by,
Aswin Balaji Thippa Ramesh
Rahul Arvind
Vishal Fulsundar

Supervised by
Dr. Amir Hossein Jafari, Ph.D.,

1. Introduction

Fatigue detection in drivers is a critical area of research due to its implications in preventing road accidents and enhancing transportation safety. This project aims to develop a real-time driver drowsiness detection system by leveraging deep learning techniques to monitor two key physiological indicators: eye closure and yawning. The system is split into two dedicated modules, one for classifying eye states as open or closed, and another for detecting yawning versus non-yawning facial expressions. Both modules are trained on a labeled dataset of facial images categorized into four classes: Open, Closed, Yawn, and No Yawn.

The technical approach involves building custom Convolutional Neural Networks (CNNs) from scratch and comparing their performance against transfer learning models based on ResNet18. Standard preprocessing and augmentation techniques are applied to enhance model generalization. To ensure interpretability and model trustworthiness, visual explanation tools like Grad-CAM and LIME are used to highlight the image regions influencing model predictions. Evaluation metrics such as accuracy, F1 score, and confusion matrices guide performance assessment. Overall, this system demonstrates the feasibility of combining eye and yawn detection for drowsiness monitoring in a reliable, real-time setup.

2. Dataset Description

The dataset used in this project was sourced from the **Kaggle notebook** [“The Detection of Eyes and Classification”](#) by **Abdelaziz Nabil**. It is designed specifically for drowsiness detection tasks and contains labeled images of human faces focused on two key regions: eyes and mouth. The dataset is divided into four distinct classes based on visual cues relevant to drowsiness detection:

Open – Images where the subject’s **eyes** are **open**

Closed – Images where the subject’s **eyes** are **closed**

Yawn – Images capturing the subject in the act of **yawning**

No Yawn – Images where the subject is **not yawning**

The dataset is organized into separate folders for training and testing.

- The training set includes 28,253 Open, 27,335 Closed, 617 Yawn, and 616 No Yawn images.
- The test set contains 7,012 Open, 6,788 Closed, 106 Yawn, and 109 No Yawn images.

All images are color JPEG files with varied lighting conditions, backgrounds, and face orientations, providing natural variability crucial for model generalization.

This dataset supports both binary classification tasks (Open vs. Closed for eye detection, Yawn vs. No Yawn for mouth detection) and multi-class classification. The diversity and realism of the

images make this dataset highly suitable for training deep learning models aimed at real-world drowsiness detection.

3. Data Preparation

The dataset used in this project is publicly available and hosted on Google Drive under the name data. It contains four labeled classes - **Open**, **Closed**, **Yawn**, and **No Yawn** each organized into separate folders within train/ and test/ directories, enabling supervised training and evaluation.

Download Instructions:

To download and unzip the dataset, follow these steps:

- **Visit the dataset link:**
https://drive.google.com/file/d/1PSWj2w2LP6Zza125W4ZmCL7t8ozEnPIA/view?usp=drive_link
- Download the ZIP file manually from your browser.
- Extract the ZIP file and ensure the folder is named data/ and placed two levels up relative to the script location. The folder structure should look like:

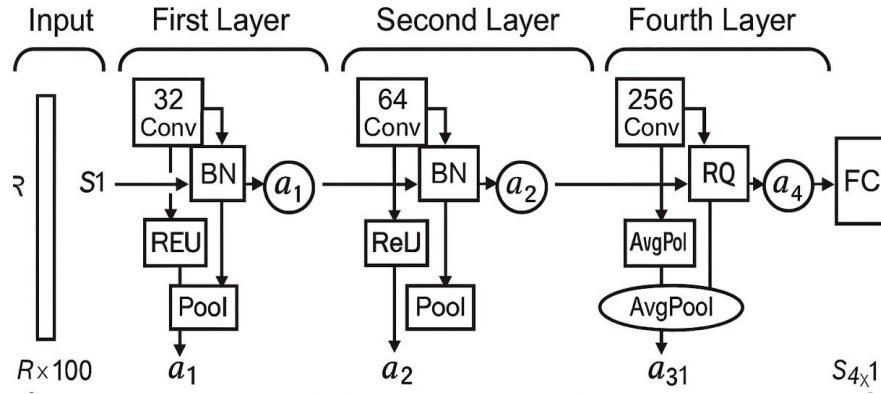
```
from torchvision import transforms
transform = transforms.Compose([
    transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),
    transforms.ToTensor(),
])
```

The dataset was already split into training and test folders for each class. We used the provided structure to load the data via ImageFolder, applying appropriate transformations such as resizing, normalization, and augmentation (random horizontal flips, rotations) to improve generalization.

4. Deep Learning Network Architecture and Training

4.1. Convolutional Neural Network (CNN) for Eye State Detection

The project implements a custom Convolutional Neural Network (CNN) to classify eye states as either 'Open' or 'Closed'. The architecture comprises multiple convolutional layers followed by pooling layers, culminating in fully connected layers for classification.



Architecture Overview:

- **Input Layer:** Receives preprocessed eye images.
- **Convolutional Layers:** Extract spatial features using filters.
- **Activation Functions:** ReLU (Rectified Linear Unit) introduces non-linearity.
- **Pooling Layers:** MaxPooling reduces spatial dimensions and computational load.
- **Fully Connected Layers:** Interpret features for classification.
- **Output Layer:** Applies Softmax activation for binary classification.

Training Details:

- **Loss Function:** CrossEntropyLoss
- **Optimizer:** Adam with a learning rate of 0.001
- **Epochs:** 25
- **Batch Size:** 32

4.2. Eye Detection using ResNet18

For eye state classification, we fine-tuned a pre-trained ResNet18 model. ResNet (Residual Network) is a deep convolutional architecture introduced by He et al. in 2015, known for its residual connections that help mitigate the vanishing gradient problem. This makes it well-suited for deeper networks and generalizes well with relatively limited data.

Model Modifications: The final fully connected layer of ResNet18 was modified to output two classes - Open and Closed:

Training Configuration:

- **Loss Function:** CrossEntropyLoss
- **Optimizer:** Adam
- **Learning Rate:** 0.0001
- **Epochs:** 25
- **Batch Size:** 30

4.3. Yawn Detection using Custom CNN

For yawn classification, a custom Convolutional Neural Network (CNN) was developed from scratch. Given the smaller dataset size and greater visual variation in yawns, a simpler and more controlled architecture allowed for better generalization and reduced overfitting.

CNN Architecture Overview:

- 4 Convolutional layers with Batch Normalization and ReLU activation
- MaxPooling layers for downsampling
- Dropout for regularization
- Fully Connected layers for final classification

Training Configuration:

- **Loss Function:** CrossEntropyLoss
- **Optimizer:** Adam
- **Learning Rate:** 0.0005
- **Epochs:** 30
- **Batch Size:** 30

This model showed better performance on yawning images, which exhibited more variation in pose and lighting. The architecture was deliberately kept compact to prevent overfitting given the limited number of yawning samples.

4.4. ResNet18 for Yawn Detection

For yawn detection, the project utilizes a pre-trained ResNet18 model, leveraging transfer learning to adapt the model to the specific task.

Modifications: Final Layer Adjustment - The original fully connected layer is replaced to match the number of output classes (Yawn/No Yawn).

Training Details:

- **Loss Function:** CrossEntropyLoss
- **Optimizer:** SGD with momentum
- **Learning Rate:** 0.001
- **Epochs:** 20
- **Batch Size:** 32

5. Analysis of Model Performance

5.1 Eye State Detection – Results and Analysis

Two models were evaluated for classifying eye states as Open or Closed: a **Custom CNN** and a **ResNet18**. Both models demonstrated strong performance, but CNN outperformed slightly in precision and misclassification reduction.

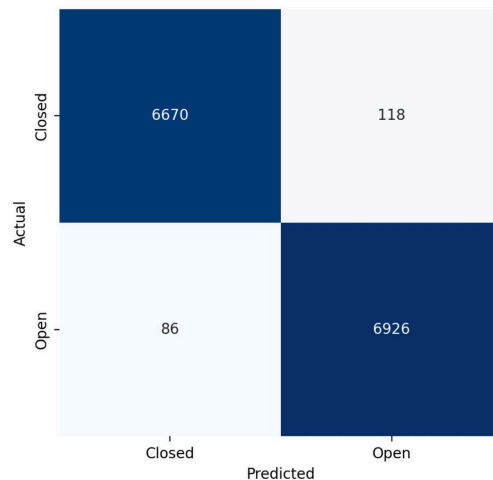
5.1.1. CNN for Eye State – Results

The CNN was designed from scratch and trained over 30 epochs. It achieved a test accuracy of 99%, with minimal misclassification.

Classification Report:

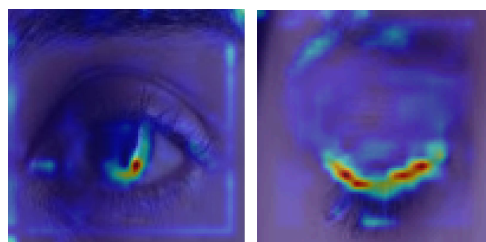
Class	Precision	Recall	F1- Score
Closed	0.99	0.99	0.99
Open	0.98	0.99	0.99

Confusion Matrix:



- The model correctly classified 6670/6788 Closed and 6926/7012 Open images.
- **Minor errors:** 118 Closed as Open and 86 Open as Closed.

Grad-CAM Visualizations: Open Eye & Closed Eye (CNN)



|Model focuses on the iris and eyelid edges when detecting open eyes.

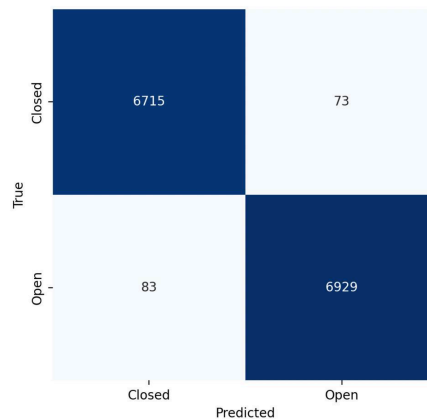
5.1.2. ResNet18 for Eye State – Results

Using pretrained ResNet18 improved performance consistency and reduced classification error further. ResNet18 achieved 99.3% accuracy with even fewer false predictions.

Classification Report:

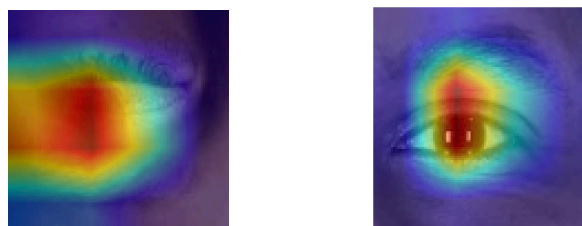
Class	Precision	Recall	F1- Score
Closed	0.99	0.99	0.99
Open	0.99	0.99	0.99

Confusion Matrix:



- Fewer misclassifications: Only 73 Closed misclassified as Open and 83 Open as Closed.
- Stronger diagonal dominance in the confusion matrix confirms better generalization.

Grad-CAM Visualizations: Open Eye & Closed Eye



- Heatmap focuses precisely on the iris and surrounding eye socket.
- Strong activation over the eyelid, showcasing spatial understanding of eye closure.

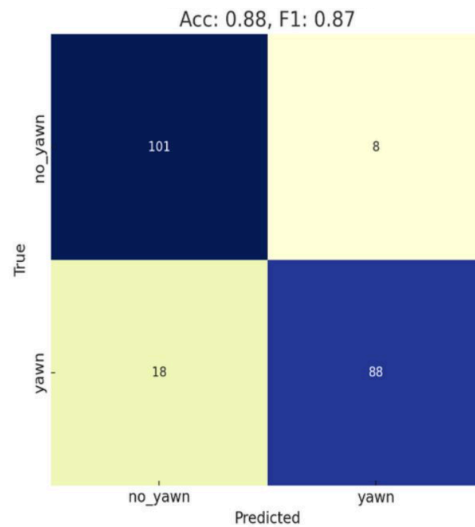
5.1.3. CNN for Yawn – Results

The custom CNN achieved decent performance in detecting yawns, despite being a simpler model. It reached 88% accuracy and an F1-score of 0.87. However, misclassifications were more frequent, particularly on the Yawn class, indicating lower generalization capability.

Classification Report:

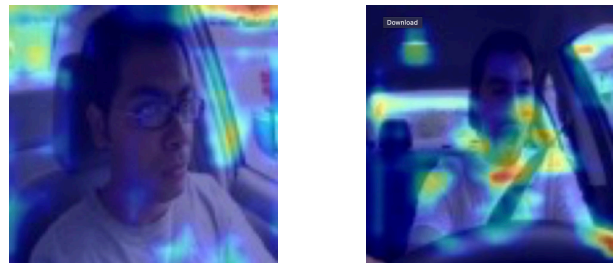
Class	Precision	Recall	F1- Score
Closed	0.89	0.87	0.88
Open	0.86	0.89	0.87

Confusion Matrix:



- Slightly weaker diagonal dominance due to small dataset and simpler model structure.

Grad-CAM Visualizations: Yawn & No Yawn



- Heatmap showed broad and unfocused activation, often extending to irrelevant areas of the face or background.
- Indicates model relied on noisy or indirect features, reducing prediction confidence.
- Activation regions were inconsistent; in some cases, non-facial areas were activated, confirming weaker attention alignment.

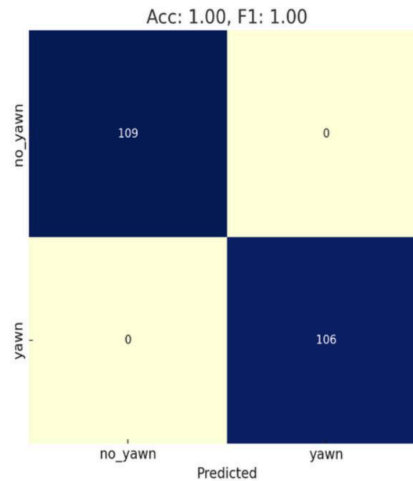
5.1.4. ResNet18 for Yawn – Results

ResNet18 significantly improved performance on yawn detection, handling class imbalance and complex facial variability much better. It achieved 100% accuracy and perfect precision/recall, with no misclassifications.

Classification Report:

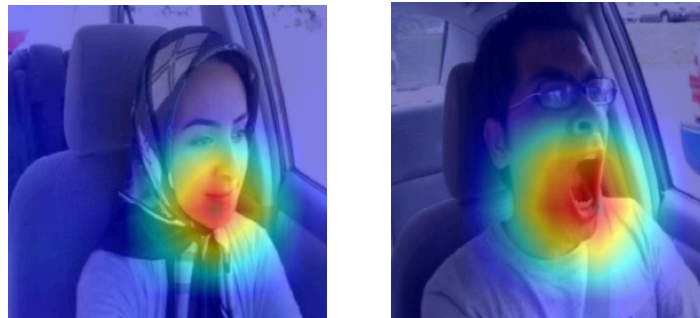
Class	Precision	Recall	F1- Score
Closed	1.00	1.00	1.00
Open	1.00	1.00	1.00

Confusion Matrix:



- All 106 yawns and 109 no yawns correctly predicted.
- Strong diagonal dominance confirms high confidence and generalization.

Grad-CAM Visualizations: Yawn & No Yawn



- Heatmap strongly centered around the open mouth and lower jaw, confirming the model learned precise yawning features.
- Focus remained centered on the lips and chin without unnecessary activation, validating correct spatial attention for non-yawning cases.

6. Live Model Prediction of Drowsiness

To make the system usable in real-world scenarios, we built a real-time application using Streamlit, OpenCV, and Media Pipe.

Webcam Feed: Captured via OpenCV.

Eye and Face Localization: Performed using MediaPipe's Face Mesh and Face Detection modules.

Eye Crop Extraction: Eyes are located using landmark indices, cropped, preprocessed, and passed to the selected model.

Face Region for Yawning: Face is cropped using bounding box detection and passed to the yawning model.

Model Inference: Each region is classified as Open/Closed or Yawn/No Yawn in real time.

Drowsiness Alerts:

- If both eyes remain closed for more than 0.5 seconds, a "Wake up" alert is triggered.
- If a yawn is detected, an audible "Yawn detected" alert is triggered immediately.

Video Processing and Alert Logic

- Integrated real time video stream from webcam & extracted **image frames** out of it.
- Integrated MediaPipe for eye and face extraction with bounding boxes, applied CNN (eye) ResNet18 (yawn) models for predictions.
- Triggered alerts based on eye closure duration (>0.5s) or detected yawns.

```
if start_detection:
    cap = cv2.VideoCapture(0)
    mp_mesh = mp.solutions.face_mesh.FaceMesh(refine_landmarks=True)
    mp_face = mp.solutions.face_detection.FaceDetection(min_detection_confidence=0.6)
    LEFT_EYE = [33, 133, 160, 159, 158, 157, 173, 155]
    RIGHT_EYE = [362, 263, 387, 386, 385, 384, 398, 382]
    eye_closed_start = None

    while cap.isOpened():
        ret, frame = cap.read()
        if not ret:
            break

        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        h, w, _ = rgb.shape
        face_results = mp_face.process(rgb)
        mesh_results = mp_mesh.process(rgb)
```

Code initializing webcam capture and MediaPipe Face Mesh/Face Detection for extracting eye and face regions from live video frames.

```
# ---- Face Landmark ----
if face_results.detections:
```

```

detection = face_results.detections[0]
box = detection.location_data.relative_bounding_box
x1, y1 = max(0, int(box.xmin * w) - 50), max(0, int(box.ymin * h) - 200)
x2, y2 = min(w, int((box.xmin + box.width) * w) + 50), min(h, int((box.ymin + box.height) * h) + 80)
face_crop = rgb[y1:y2, x1:x2]
if face_crop.size > 0:
    tensor, raw = preprocess_image(face_crop, yawn_size, yawn_norm)
    label, idx = predict(yawn_model, tensor, YAWN_CLASSES)
    cam = overlay_gradcam_on_frame(raw, tensor, yawn_model, idx, yawn_model_choice == "ResNet18")
    yawn_disp.image([raw, cam], caption=["Yawn Input", "Yawn Grad-CAM"], width=220)
    if label == "yawn": beep_alert("Yawn detected")
    cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 0, 255), 2)
    cv2.putText(frame, f"Yawn: {label}", (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0,0,255),
2)

```

Code for extracting face region from detected bounding box, preprocessing it, predicting yawn status with the model, and triggering an alert if a yawn is detected.

```

# ---- Eye Landmark ----
if mesh_results.multi_face_landmarks:
    landmarks = mesh_results.multi_face_landmarks[0].landmark
    closed_count = 0
    for i, eye_pts in enumerate([LEFT_EYE, RIGHT_EYE]):
        x = [int(landmarks[i].x * w) for i in eye_pts]
        y = [int(landmarks[i].y * h) for i in eye_pts]
        x1, y1 = max(min(x) - 30, 0), max(min(y) - 50, 0)
        x2, y2 = min(max(x) + 30, w), min(max(y) + 40, h)
        eye_crop = rgb[y1:y2, x1:x2]
        if eye_crop.size > 0:
            tensor, raw = preprocess_image(eye_crop, eye_size, eye_norm)
            label, idx = predict(eye_model, tensor, EYE_CLASSES)
            cam = overlay_gradcam_on_frame(raw, tensor, eye_model, idx, eye_model_choice == "ResNet18")
            if i == 0:
                left_eye_disp.image([raw, cam], caption=["Left Eye", "Grad-CAM"], width=220)
            else:
                right_eye_disp.image([raw, cam], caption=["Right Eye", "Grad-CAM"], width=220)
            if label == "Closed": closed_count += 1
            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv2.putText(frame, f'{"Left","Right"}[i]: {label}', (x1, y1 - 10), cv2.FONT_HERSHEY_SIMPLEX,
0.6, (0,255,0), 2)

```

Code for extracting left and right eye regions from face landmarks, preprocessing each eye, predicting eye state, and annotating the frame with predictions and bounding boxes.

```

if closed_count == 2:
    if eye_closed_start is None:
        eye_closed_start = time.time()
    elif time.time() - eye_closed_start > 0.5:
        beep_alert("Wake up")
        eye_closed_start = None
    else:
        eye_closed_start = None

```

Code logic to trigger a “Wake up” alert if both eyes remain closed for more than 0.5 seconds.

Model Explainability with Grad-CAM:

- Performed a forward and backward pass to calculate gradients for the predicted class.
- Computed Grad-CAM heatmap by weighting activation maps with pooled gradients.
- Normalized and resized the heatmap to match input image dimensions.
- Overlayed the colored heatmap onto the original input image for visualizing important regions.

```
def overlay_gradcam_on_frame(crop, tensor_input, model, class_idx, is_resnet=False):
    gradients, activations = [], []

    def forward_hook(module, input, output):
        activations.append(output)

    def backward_hook(module, grad_input, grad_output):
        gradients.append(grad_output[0])

    for module in model.modules():
        if isinstance(module, nn.Conv2d):
            last_conv = module

    h1 = last_conv.register_forward_hook(forward_hook)
    h2 = last_conv.register_backward_hook(backward_hook)

    model.zero_grad()
    output = model(tensor_input)
    output[0, class_idx].backward()
    grads = gradients[0]
    acts = activations[0]
    pooled_grads = torch.mean(grads, dim=(0, 2, 3))
    for i in range(acts.shape[1]):
        acts[:, i, :, :] *= pooled_grads[i]

    heatmap = torch.mean(acts, dim=1).squeeze()
    heatmap = F.relu(heatmap)
    heatmap /= torch.max(heatmap + 1e-6)
    heatmap = heatmap.detach().cpu().numpy()

    heatmap = cv2.resize(heatmap, (crop.shape[1], crop.shape[0]))
    heatmap = np.uint8(255 * heatmap)
    heatmap_colored = cv2.applyColorMap(heatmap, cv2.COLORMAP_JET)
    return cv2.addWeighted(crop, 0.6, heatmap_colored, 0.4, 0)
```

Function to generate and overlay Grad-CAM heatmap on input image.

7. Drawbacks

While the system successfully detects drowsiness cues like eye closure and yawning in real time, several limitations remain that affect generalizability, robustness, and real-world deployment. This version is optimized for frontal face detection under controlled lighting, future versions can expand to handle head rotation, varied environments, and continuous user calibration.

Frontal Face Dependency

- The system relies on frontal face positioning for accurate landmark detection.
- Head tilts, side profiles, or partial occlusion (e.g., sunglasses) significantly reduce detection accuracy.

Lighting Sensitivity

- Performance drops in low-light or high-glare conditions, which are common in nighttime or sunlight scenarios. No adaptive exposure correction or infrared support is implemented.
- There is no consideration for blink rate trends or cumulative fatigue signs.

No Temporal Modeling

- The system lacks temporal awareness (e.g., RNNs, LSTMs), which could track fatigue over time.
- It evaluates each frame independently, missing sequential context that could improve prediction robustness.

8. Summary and Conclusion

This project aimed to develop a real-time Driver Drowsiness Detection System capable of identifying key fatigue indicators: eye closure and yawning. Using a combination of custom CNNs and ResNet18 models, along with MediaPipe for facial landmark detection and Streamlit for deployment, the system processes webcam input and issues audio alerts when drowsiness is detected.

We trained models on a curated dataset of ~70,000 images sourced from multiple Kaggle datasets, balancing between Open/Closed eyes and Yawn/No Yawn facial expressions. The system supports dynamic model selection and performs accurate binary classification in real time.

Our key findings include:

- The Custom CNN performed with high precision in eye detection, especially in focusing tightly on the eyelid and iris region.
- ResNet18, pre-trained on ImageNet, achieved 100% accuracy in yawn detection, demonstrating strong generalization even with limited data.

The live deployment successfully triggered alerts when both eyes remained closed for over 0.5 seconds or when yawning was detected, with minimal false positives.

Despite the system's strengths, it has several limitations, including a dependency on frontal face orientation, sensitivity to lighting, and lack of temporal modeling. These constraints will be addressed in future iterations by incorporating head-pose tolerance, larger and more diverse training datasets, and RNN-based sequence modeling to detect patterns over time.

Overall, the project demonstrates a practical, interpretable, and responsive driver monitoring system that lays the groundwork for future integration into real-world vehicle safety platforms.

9. References

1. He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep Residual Learning for Image Recognition. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 770–778. <https://doi.org/10.1109/CVPR.2016.90>

Used as the foundation for the ResNet18 architecture applied in this project.

2. Kingma, D. P., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. In International Conference on Learning Representations (ICLR). <https://arxiv.org/abs/1412.6980>

Optimizer used in training both CNN and ResNet models.

3. T. Soukupová and J. Čech. (2016). Real-Time Eye Blink Detection Using Facial Landmarks. In 21st Computer Vision Winter Workshop.

Key reference for using facial landmarks in eye state detection.

4. Google MediaPipe. (2023). MediaPipe: Cross-platform, customizable ML solutions for live and streaming media. <https://google.github.io/mediapipe/>

Used for real-time face and eye landmark detection.

5. Selvaraju, R. R., Cogswell, M., Das, A., et al. (2017). Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. Proceedings of the IEEE International Conference on Computer Vision (ICCV), 618–626. <https://doi.org/10.1109/ICCV.2017.74>

Referenced for attention visualization (if needed for prior versions or future extensions).

6. OpenCV Library. (2023). Open Source Computer Vision Library. <https://opencv.org/>

Used for image processing, webcam feed capture, and drawing bounding boxes.